

3YC AFRICANAS

Sistema de Gestão de Oficina Mecânica

Documento de Especificação de Requisitos de Software

(ERS — Engenharia de Software)

Desenvolvido por:
Miguel Fernando
Cleusem
Nicolau

1. Introdução

1.1 Propósito

O presente documento tem como propósito descrever os requisitos funcionais e não funcionais do Sistema de Gestão de Oficina Mecânica 3YC AFRICANAS, desenvolvido para modernizar e automatizar a gestão operacional de oficinas mecânicas em Angola.

Este documento serve como referência técnica para desenvolvedores, gestores e partes interessadas, detalhando a arquitectura, funcionalidades, modelo de dados e casos de uso do sistema.

1.2 Escopo

O sistema 3YC AFRICANAS é uma aplicação web completa que abrange:

- Gestão de clientes e veículos
- Controlo de ordens de serviço
- Gestão de estoque de peças e produtos
- Controlo de funcionários e perfis de acesso
- Módulo financeiro com pagamentos e emissão de faturas
- Geração de relatórios gerenciais em PDF
- Dashboard com indicadores em tempo real e gráficos

1.3 Definições e Abreviações

Termo	Definição
OS	Ordem de Serviço — documento que regista o serviço a realizar num veículo
ERS	Especificação de Requisitos de Software
CRUD	Create, Read, Update, Delete — operações básicas de base de dados
API	Application Programming Interface — interface de comunicação entre sistemas
IVA	Imposto sobre o Valor Acrescentado (14% — Angola)
PHP	Hypertext Preprocessor — linguagem de programação do back-end
MySQL	Sistema de gestão de base de dados relacional
JSON	JavaScript Object Notation — formato de troca de dados
PDF	Portable Document Format — formato de documento para impressão
Dashboard	Painel de controlo com indicadores visuais e gráficos

2. Descrição Geral

2.1 Perspectiva do Produto

O sistema 3YC AFRICANAS é uma aplicação web standalone desenvolvida com tecnologias modernas, concebida para funcionar em ambiente local (XAMPP/Apache) com base de dados MySQL. A aplicação é composta por:

- Front-end: HTML5, CSS3, JavaScript puro com Bootstrap 5 e Chart.js
- Back-end: PHP 8.0+ com PDO para acesso à base de dados
- Base de dados: MySQL 5.7+ com modelo relacional normalizado
- Arquitectura: REST API com separação clara entre front-end e back-end

2.2 Funções Principais

O sistema oferece as seguintes funções principais agrupadas por módulo:

Módulo	Função Principal	Perfil de Acesso
Dashboard	Indicadores KPI, gráficos de ordens e faturamento	Admin
Clientes	Cadastro, edição, histórico de atendimentos	Admin
Veículos	Registo de veículos associados a clientes	Admin
Funcionários	Gestão de técnicos e perfis de acesso	Admin
Ordens de Serviço	Criação, acompanhamento e conclusão de OS	Admin, Funcionário
Estoque	Controlo de peças com alertas de nível mínimo	Admin, Funcionário
Pagamentos	Registo de pagamentos por múltiplos métodos	Admin
Faturas	Emissão e impressão de faturas com IVA 14%	Admin
Relatórios	Relatórios por período exportáveis em PDF	Admin
Utilizadores	Gestão de contas e perfis do sistema	Admin

2.3 Restrições

- O sistema requer servidor web Apache com PHP 8.0 ou superior
- Requer MySQL 5.7+ ou MariaDB 10.3+
- A aplicação não suporta multi-tenancy (uma instância por oficina)
- Não inclui integração nativa com sistemas de pagamento externos
- Não possui aplicação móvel nativa (apenas web responsivo)

2.4 Suposições e Dependências

- O utilizador dispõe de um servidor local (XAMPP, WAMP ou Laragon)
- Existe conectividade com a internet para carregar CDNs (Bootstrap, FontAwesome, Chart.js)
- Os utilizadores têm conhecimentos básicos de navegação web
- A oficina opera em moeda Kwana (Kz) com IVA de 14% (Angola)

3. Requisitos

3.1 Requisitos Funcionais

RF01 — Autenticação e Controlo de Acesso

ID	Descrição	Prioridade
RF01.1	O sistema deve permitir login com email e senha	Alta
RF01.2	O sistema deve suportar três perfis: Admin, Funcionário e Cliente	Alta
RF01.3	O sistema deve redirecionar para o dashboard após login	Alta
RF01.4	O sistema deve permitir registo de novos utilizadores	Média
RF01.5	O sistema deve manter sessão activa durante a utilização	Alta
RF01.6	O sistema deve efectuar logout e redirecionar para a página inicial	Alta

RF02 — Gestão de Clientes

ID	Descrição	Prioridade
----	-----------	------------

RF02.1	Cadastrar clientes com nome, email, telefone, endereço e NIF	Alta
RF02.2	Editar dados de clientes existentes	Alta
RF02.3	Remover (desactivar) clientes do sistema	Média
RF02.4	Listar clientes com filtro por nome, email ou NIF	Alta
RF02.5	Visualizar histórico de veículos e ordens de serviço do cliente	Média
RF02.6	Registar total acumulado gasto pelo cliente	Baixa

RF03 — Gestão de Veículos

ID	Descrição	Prioridade
RF03.1	Registar veículo associado a um cliente (modelo, placa, cor, ano)	Alta
RF03.2	Editar dados de veículos	Alta
RF03.3	Remover veículos do sistema	Média
RF03.4	Listar veículos com filtro por placa, modelo ou cliente	Alta
RF03.5	Actualizar automaticamente o status do veículo ao criar/concluir OS	Média

RF04 — Ordens de Serviço

ID	Descrição	Prioridade
RF04.1	Criar OS com cliente, veículo, funcionário, serviços e peças	Alta
RF04.2	Calcular automaticamente o valor total da OS	Alta
RF04.3	Actualizar o status da OS (Pendente, Em Andamento, Concluída, Cancelada)	Alta
RF04.4	Listar OS com filtro por status, cliente ou código	Alta
RF04.5	Visualizar detalhes completos de uma OS	Alta
RF04.6	Baixar automaticamente o estoque ao criar uma OS com peças	Alta
RF04.7	Gerar código único sequencial para cada OS (OS0001, OS0002...)	Alta

RF05 — Controlo de Estoque

ID	Descrição	Prioridade
RF05.1	Cadastrar produtos com nome, categoria, quantidade e preço	Alta
RF05.2	Definir quantidade mínima com alerta automático de estoque baixo	Alta
RF05.3	Ajustar manualmente o estoque (entrada/saída)	Média
RF05.4	Listar produtos com indicador visual de status (OK / Estoque Baixo)	Alta
RF05.5	Mostrar badge de notificação com contagem de produtos em alerta	Média

RF06 — Pagamentos e Faturação

ID	Descrição	Prioridade
RF06.1	Registar pagamento associado a uma OS com método de pagamento	Alta
RF06.2	Suportar métodos: Dinheiro, Transferência, Cartão, M-Pesa, Cheque	Alta
RF06.3	Gerar fatura automaticamente ao registar pagamento	Alta
RF06.4	Calcular IVA de 14% sobre o valor dos serviços	Alta
RF06.5	Visualizar fatura detalhada com serviços, peças, subtotal, IVA e total	Alta
RF06.6	Imprimir/exportar fatura em PDF directamente pelo browser	Alta
RF06.7	Marcar OS como Concluída automaticamente ao registar pagamento	Alta

RF07 — Relatórios

ID	Descrição	Prioridade
RF07.1	Gerar relatório por período (data início / data fim)	Alta
RF07.2	Mostrar resumo: total ordens, receita, concluídas, em andamento	Alta
RF07.3	Listar top 5 clientes por valor gasto no período	Média
RF07.4	Mostrar produtos com estoque crítico	Média
RF07.5	Listar todas as ordens do período com detalhes	Alta
RF07.6	Exportar relatório em PDF via impressão do browser	Alta

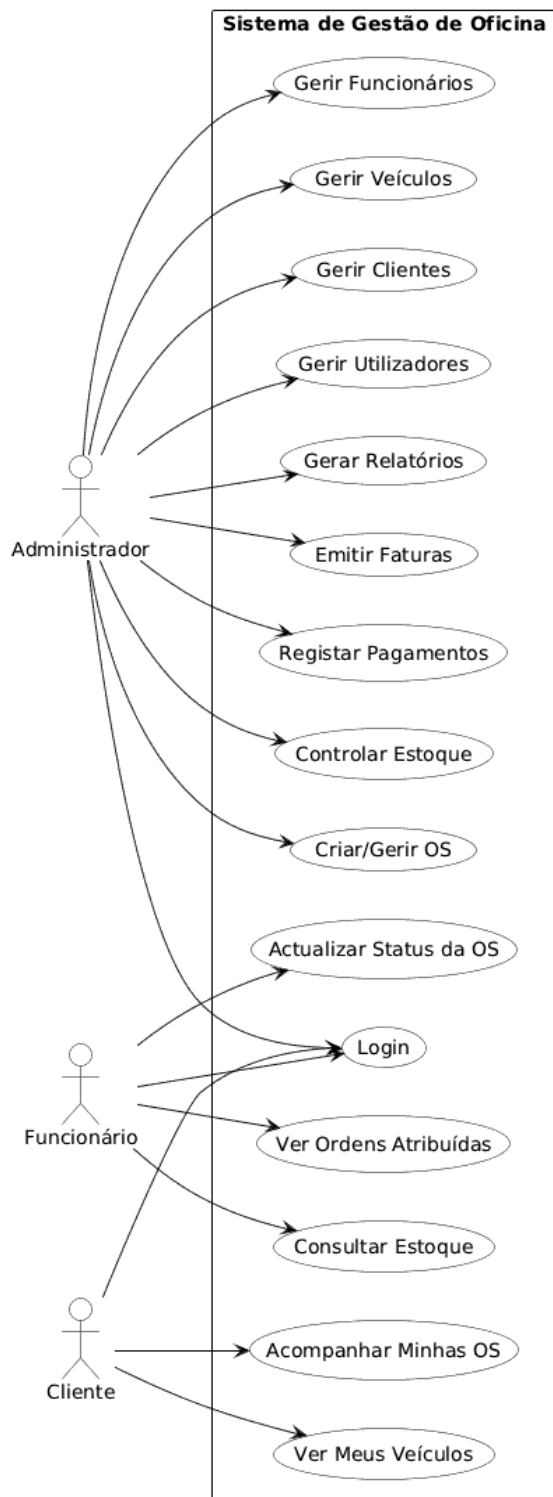
3.2 Requisitos Não Funcionais

ID	Categoria	Descrição
RNF01	Desempenho	As páginas devem carregar em menos de 3 segundos em rede local
RNF02	Usabilidade	Interface responsiva compatível com desktop, tablet e telemóvel
RNF03	Segurança	Sessões geridas via sessionStorage com redirecionamento automático
RNF04	Compatibilidade	Compatível com Chrome, Firefox, Edge e Safari modernos
RNF05	Manutenibilidade	Código organizado em módulos separados (PHP + JS)
RNF06	Portabilidade	Funciona em Windows, Linux e macOS com XAMPP/LAMP
RNF07	Escalabilidade	Arquitetura REST permite futura integração com aplicações móveis
RNF08	Acessibilidade	Suporte a modo claro e escuro com toggle persistente

4. Modelagem do Sistema

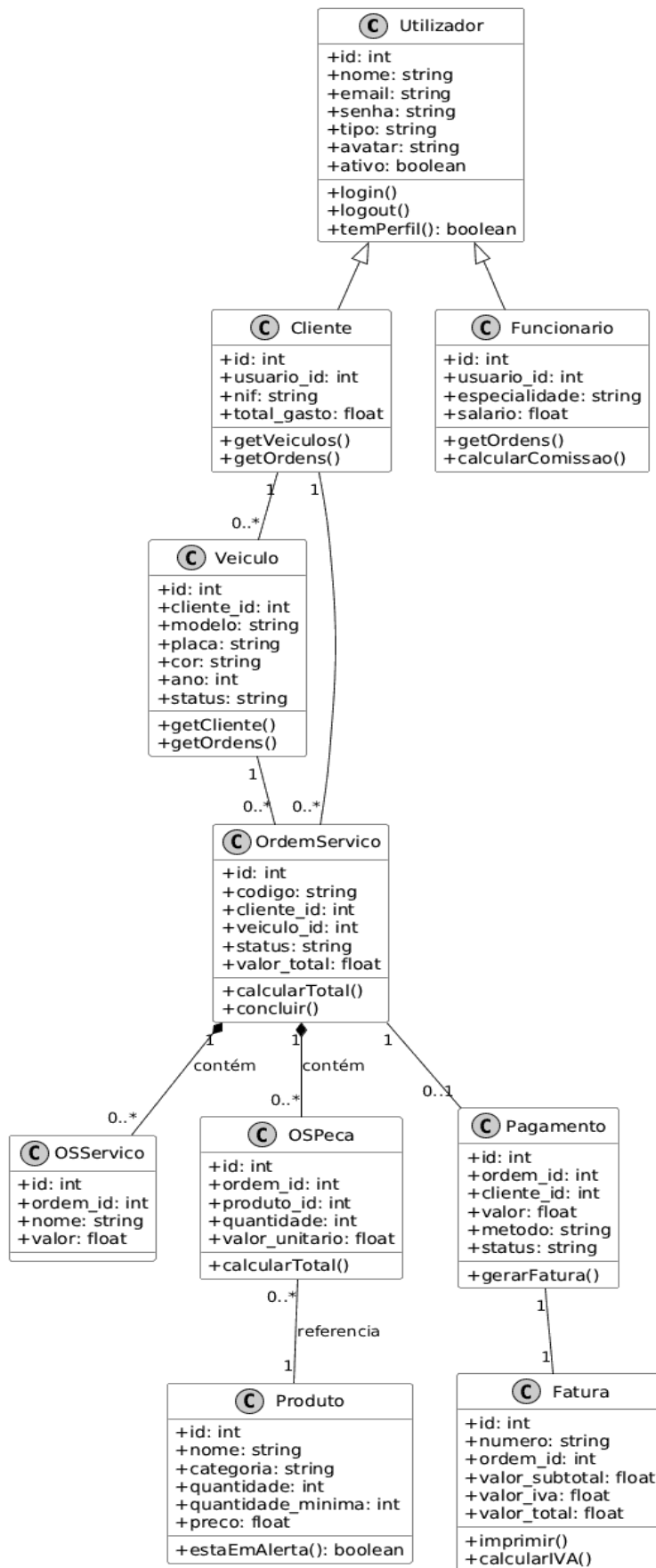
4.1 Diagrama de Casos de Uso

O sistema possui três actores principais com os seguintes casos de uso:



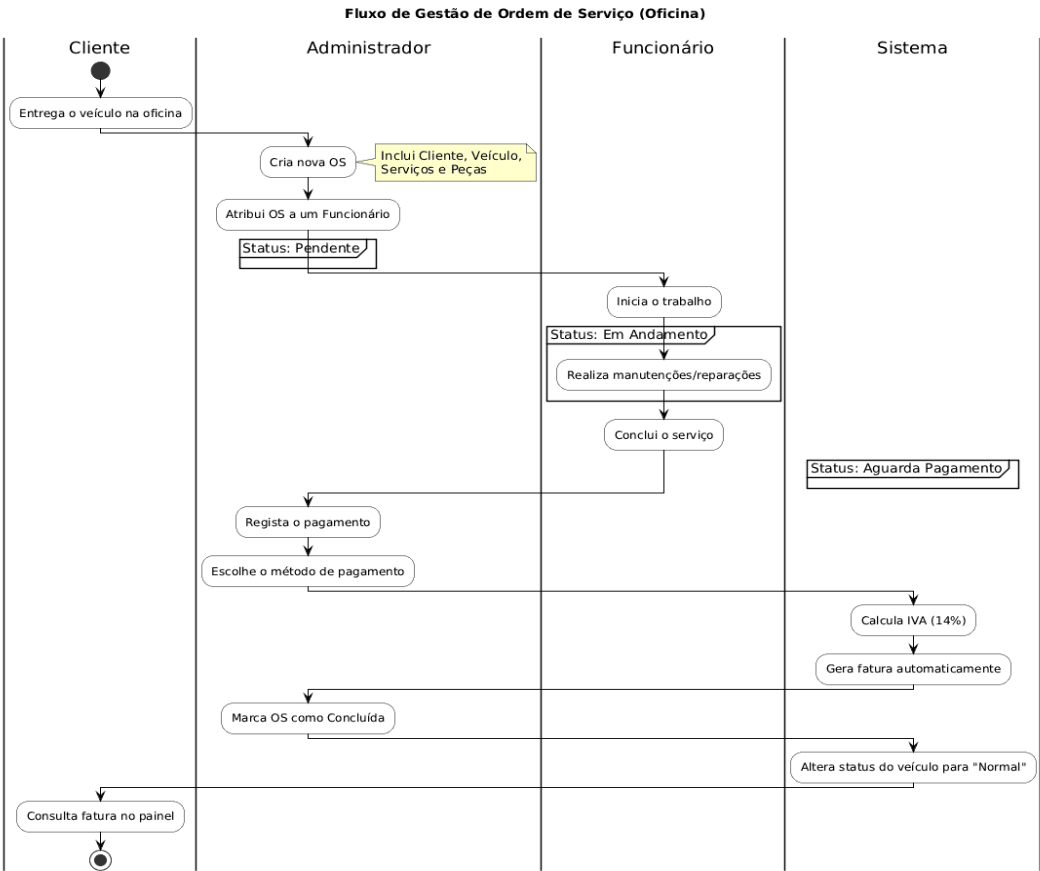
Actor	Casos de Uso
Administrador	Login, Gerir Clientes, Gerir Veículos, Gerir Funcionários, Criar/Gerir OS, Controlar Estoque, Registrar Pagamentos, Emitir Faturas, Gerar Relatórios, Gerir Utilizadores
Funcionário	Login, Ver Ordens Atribuídas, Actualizar Status da OS, Consultar Estoque
Cliente	Login, Ver Meus Veículos, Acompanhar Minhas Ordens de Serviço

4.2 Diagrama de Classes — Principais Entidades



Classe	Atributos Principais	Métodos/Relações
Utilizador	id, nome, email, senha, tipo, avatar, ativo	login(), logout(), temPerfil()
Cliente	id, usuario_id, nome, email, telefone, nif, total_gasto	getVeiculos(), getOrdens()
Funcionario	id, usuario_id, nome, email, especialidade, salario	getOrdens(), calcularComissao()
Veiculo	id, cliente_id, modelo, placa, cor, ano, status	getCliente(), getOrdens()
OrdemServico	id, codigo, cliente_id, veiculo_id, status, valor_total	calcularTotal(), concluir()
OrdemServico (Serviço)	id, ordem_id, nome, valor	—
OrdemServico (Peça)	id, ordem_id, produto_id, quantidade, valor_unitario	calcularTotal()
Produto	id, nome, categoria, quantidade, quantidade_minima, preco	estaEmAlerta()
Pagamento	id, ordem_id, cliente_id, valor, metodo, status	gerarFatura()
Fatura	id, numero, ordem_id, valor_subtotal, valor_iva, valor_total	imprimir(), calcularIVA()

4.3 Diagrama de Actividades — Fluxo de Ordem de Serviço



O fluxo principal de uma Ordem de Serviço segue as seguintes etapas:

- 1. Cliente traz veículo à oficina
- 2. Admin cria nova OS com cliente, veículo, serviços e peças previstas
- 3. OS é atribuída a um Funcionário — status: Pendente
- 4. Funcionário inicia o trabalho — status: Em Andamento
- 5. Funcionário conclui o serviço — status: aguarda pagamento
- 6. Admin regista o pagamento (escolhe método)
- 7. Sistema gera fatura automaticamente com IVA 14%
- 8. OS é marcada como Concluída — veículo volta a status Normal
- 9. Cliente pode consultar a fatura no seu painel

5. Design e Arquitectura

5.1 Arquitectura do Sistema

Frontend

O front-end é composto por ficheiros HTML5, CSS3 e JavaScript puro, servidos directamente pelo Apache:

- index.html — Página inicial / landing page com modais de login e cadastro
- dashboard.html — Painel principal com sidebar, topbar e área de conteúdo dinâmico
- assets/js/api.js — Cliente HTTP que comunica com a REST API
- assets/js/auth.js — Lógica de autenticação e gestão de sessão
- assets/js/dashboard.js — Lógica de todas as páginas do dashboard

Backend

O back-end é uma REST API em PHP 8.0+ organizada em endpoints independentes:

Endpoint	Métodos	Funcionalidade
api/auth.php	GET, POST	Login, logout, cadastro, verificação de sessão
api/clientes.php	GET, POST, PUT, DELETE	CRUD completo de clientes
api/veiculos.php	GET, POST, PUT, DELETE	CRUD completo de veículos
api/funcionarios.php	GET, POST, PUT, DELETE	CRUD completo de funcionários
api/ordens.php	GET, POST, PUT, DELETE	CRUD de ordens de serviço
api/produtos.php	GET, POST, PUT, DELETE	CRUD de produtos/estoque
api/pagamentos.php	GET, POST, DELETE	Registo e gestão de pagamentos
api/faturas.php	GET	Listagem, detalhe e relatórios de faturas
api/dashboard.php	GET	Estatísticas e dados para gráficos
api/usuarios.php	GET, POST, PUT, DELETE	Gestão de utilizadores do sistema

Banco de Dados

A base de dados MySQL segue o modelo relacional normalizado na 3ª Forma Normal (3FN) com as seguintes tabelas:

- usuarios — Contas de acesso ao sistema
- clientes — Dados dos clientes da oficina
- funcionarios — Dados dos técnicos/mecânicos
- veiculos — Veículos registados por cliente
- ordens_servico — Ordens de serviço principais
- ordem_servicos — Serviços de cada OS (N:1)
- ordem_pecas — Peças utilizadas em cada OS (N:1)
- produtos — Catálogo de peças e produtos em estoque
- movimentos_estoque — Histórico de entradas e saídas
- pagamentos — Pagamentos efectuados
- faturas — Faturas emitidas com IVA

Padrão Arquitectural

O sistema segue o padrão MVC simplificado adaptado para aplicações web sem framework:

- Model: Ficheiros PHP da API que acedem directamente à BD via PDO
- View: Ficheiros HTML + JavaScript que renderizam a interface
- Controller: Funções JavaScript no dashboard.js que coordenam Model e View

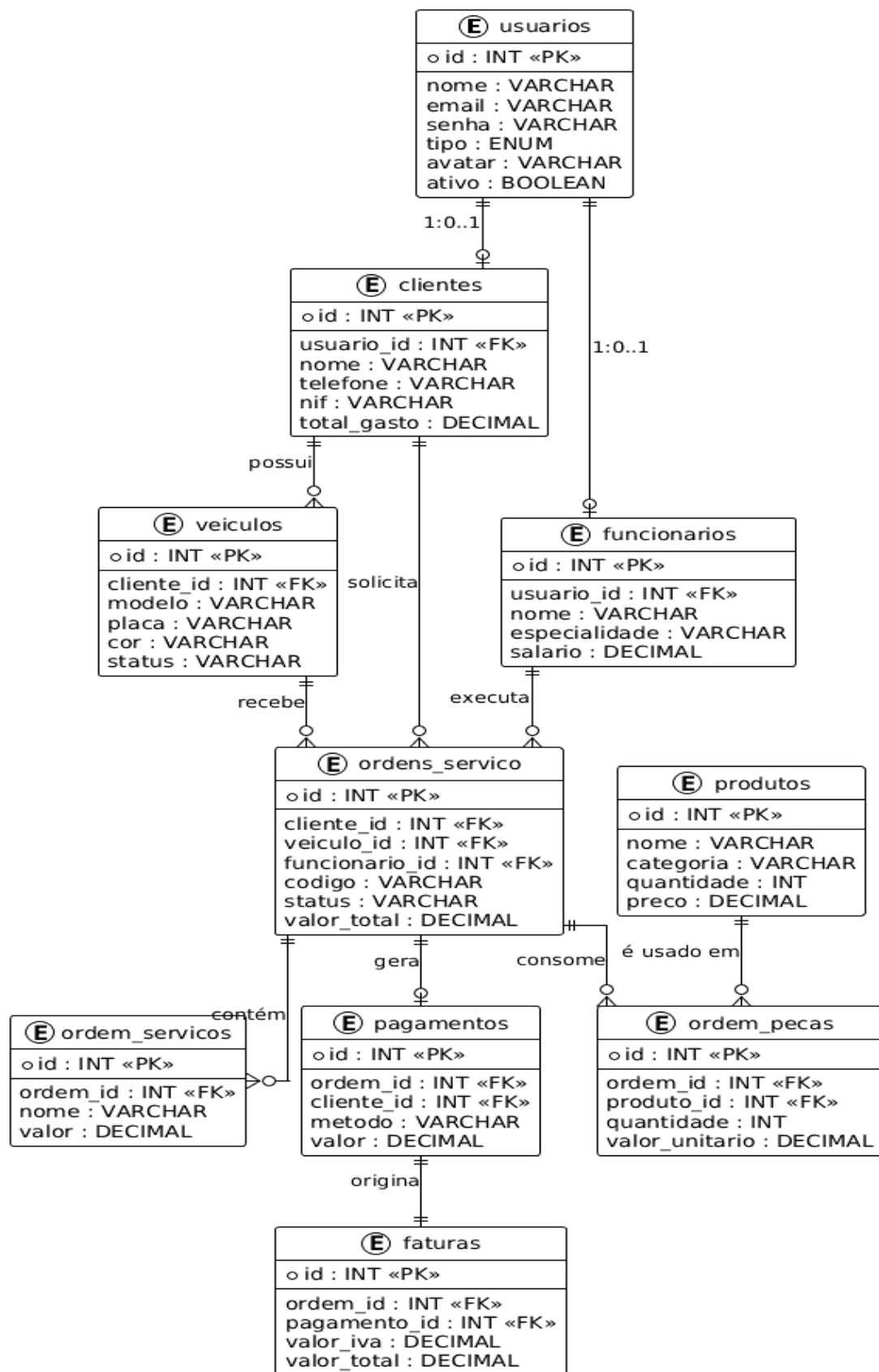
5.2 Protótipo da Interface

A interface do sistema foi concebida com as seguintes características de design:

- Paleta: Tema escuro (padrão) e claro com toggle persistente
- Tipografia: Syne (títulos) + Plus Jakarta Sans (corpo)
- Cores principais: Indigo (#6366f1) + Cyan (#06b6d4) + Âmbar (#f59e0b)
- Sidebar fixa com navegação por perfil (Admin / Funcionário / Cliente)
- Cards KPI com gráficos de barras e doughnut via Chart.js 4
- Tabelas com paginação e filtros em tempo real
- Modais Bootstrap 5 para formulários CRUD
- Sistema de notificações toast com animações CSS

5.3 Diagrama Entidade-Relacionamento (DER)

O modelo de dados do sistema 3YC AFRICANAS é composto por 11 tabelas com os seguintes relacionamentos:



Entidade	PK	FKs	Atributos Principais	Cardinalidade
usuarios	id	—	nome, email, senha, tipo, avatar, ativo	1 utilizador → 0..1 cliente OU funcionário
clientes	id	usuario_id → usuarios	nome, email, telefone, nif, total_gasto, data_cadastro	1 cliente → N veículos, N ordens
funcionarios	id	usuario_id → usuarios	nome, email, telefone, especialidade, salario	1 funcionário → N ordens
veiculos	id	cliente_id → clientes	modelo, placa, cor, ano, ultima_visita, status	1 veículo → N ordens
ordens_servico	id	cliente_id → clientes, veiculo_id → veiculos, funcionario_id → funcionarios	codigo, status, valor_total, data_entrada, data_saida	1 OS → N serviços, N peças, 0..1 pagamento
ordem_servicos	id	ordem_id → ordens_servico	nome, valor	N serviços → 1 OS
ordem_pecas	id	ordem_id → ordens_servico, produto_id → produtos	nome, quantidade, valor_unitario, valor_total	N peças → 1 OS, N peças → 1 produto
produtos	id	—	nome, categoria, quantidade, quantidade_minima, preco, ativo	1 produto → N movimentos, N ordem_pecas
movimentos_estoque	id	produto_id → produtos, ordem_id → ordens_servico	tipo, quantidade, motivo	N movimentos → 1 produto
pagamentos	id	ordem_id → ordens_servico, cliente_id → clientes	valor, metodo, status, referencia, data_pagamento	1 pagamento → 1 OS, 1 cliente
faturas	id	ordem_id → ordens_servico, cliente_id → clientes, pagamento_id → pagamentos	numero, valor_subtotal, valor_iva, valor_total, status	1 fatura → 1 OS, 1 cliente, 1 pagamento

Relações Principais do DER

Relação	Tipo	Descrição
utilizadores → clientes	1:0..1	Um utilizador pode ser (ou não) um cliente
utilizadores → funcionarios	1:0..1	Um utilizador pode ser (ou não) um funcionário
clientes → veiculos	1:N	Um cliente pode ter vários veículos
clientes → ordens_servico	1:N	Um cliente pode ter várias ordens
veiculos → ordens_servico	1:N	Um veículo pode ter várias ordens
funcionarios → ordens_servico	1:N	Um funcionário pode ser atribuído a várias ordens
ordens_servico → ordem_servicos	1:N	Uma OS tem vários serviços
ordens_servico → ordem_pecas	1:N	Uma OS tem várias peças utilizadas
produtos → ordem_pecas	1:N	Um produto pode estar em várias OS
produtos → movimentos_estoque	1:N	Um produto tem histórico de movimentos
ordens_servico → pagamentos	1:0..1	Uma OS tem no máximo um pagamento
ordens_servico → faturas	1:0..1	Uma OS tem no máximo uma fatura
pagamentos → faturas	1:0..1	Um pagamento gera uma fatura

6. Testes

6.1 Tipos de Teste

Teste de Unidade

Testes individuais de cada endpoint da API PHP:

- Validação dos campos obrigatórios (nome, email, etc.)
- Verificação das respostas JSON (sucesso e erro)
- Testes de autenticação (login válido/inválido)
- Verificação dos cálculos de IVA e totais

Teste de Integração

Testes do fluxo completo de operações:

- Criação de cliente → veículo → OS → pagamento → fatura
- Baixa automática de estoque ao criar OS com peças
- Actualização do total gasto do cliente após pagamento
- Transição de status do veículo (Normal → Em oficina → Normal)

Teste de Aceitação

Validação com utilizadores reais dos cenários principais:

- Login com perfil Admin e acesso a todas as funcionalidades
- Login com perfil Funcionário e restrição ao menu administrativo
- Login com perfil Cliente e visualização apenas das suas OS e veículos
- Emissão e impressão de fatura em PDF

Teste de Usabilidade

- Tempo de carregamento das páginas (< 3 segundos)
- Responsividade em diferentes tamanhos de ecrã
- Funcionamento do modo claro/escuro
- Funcionamento da pesquisa global em tempo real
- Correcta exibição dos gráficos e indicadores KPI

7. Implantação

O sistema é implantado em ambiente local seguindo os passos:

Passo	Acção	Detalhe
1	Instalar XAMPP	Versão 8.0+ com Apache e MySQL activados
2	Copiar ficheiros	Extrair ZIP para C:\xampp\htdocs\bm_backend\
3	Criar base de dados	phpMyAdmin → Novo → oficina_3yc → utf8mb4
4	Importar SQL	Executar config/database.sql + config/criar_tabelas_novas.sql
5	Configurar conexão	Editar config/database.php com credenciais MySQL
6	Aceder ao sistema	http://localhost/bm_backend/public/
7	Primeiro login	admin@3yc.com / password

Requisitos mínimos de hardware para instalação local:

- Processador: Dual-core 2.0 GHz ou superior
- Memória RAM: 4 GB mínimo (8 GB recomendado)
- Armazenamento: 500 MB para o sistema (excluindo dados)
- Sistema Operativo: Windows 10+, macOS 10.15+, Ubuntu 20.04+

8. Manutenção e Evolução

8.1 Manutenção Correctiva

Para corrigir erros no sistema:

- Verificar os logs do Apache em C:\xampp\apache\logs\error.log
- Usar o Console do browser (F12) para erros de JavaScript
- Verificar a conectividade da API em http://localhost/bm_backend/api/auth.php?action=sessao
- Fazer backup da base de dados antes de qualquer alteração

8.2 Manutenção Evolutiva — Funcionalidades Futuras

Funcionalidade	Prioridade	Complexidade
Aplicação móvel (React Native)	Alta	Alta
Integração com M-Pesa API Angola	Alta	Média
Envio de faturas por email (SMTP)	Alta	Baixa
Módulo de agendamento de serviços	Média	Média
Integração com sistemas contabilísticos	Média	Alta
Multi-filial (várias oficinas)	Baixa	Alta
Portal do cliente online	Média	Média
Sistema de avaliação de serviços	Baixa	Baixa
Backup automático da base de dados	Alta	Baixa
Suporte a múltiplas moedas	Baixa	Média